```
# -*- coding: utf-8 -*-
"""
قابلیت «سلف» روبی.

- پشتیبانی از پنل مدیریت (🤖 سلف) با آیدی عددی کاربر، سلف رو برای اون کاربر فعال/غیرفعال می‌کنه.
- کاربرِ فعال‌شده با /self (یا نوشتن «سلف») تنظیم می‌کنه و تا ۳ کلمه تنظیم می‌کنه:
  هر کلمه = (کلمه، گپ، تایمر، جای دکمه: چپ/وسط/راست/خالی).
- دستورهای نقطه‌ای داخل چت از اکانت کاربر: .پنل  .کلمه فلان  .زمان فلان  .دکمه فلان  .گپ [اسم گروه]
- حالت خودکار: اسلاتی که کلمه + گپ + تایمر (=> ۵ ثانیه) داره، هر تایمر ثانیه خودش کلمه رو توی گپ می‌گه و
دکمه‌ی انتخابی رو می‌زنه.
- وقتی کاربر از اکانت خودش دقیقاً اون کلمه رو توی همون گپ بفرسته، (حالت دستی قدیمی، فقط اگه تایمر صفر باشه)
بعد از تایمر، سلف روی دکمه‌ی انتخاب‌شده‌ی
  پیام ربات (ریپلای یا آخرین پیام دکمه‌دار ربات‌های دیگه) کلیک می‌کنه. روی ربات روباهیو هیچ‌وقت کلیک نمی‌کنه.

- ساعت زنده کنار اسم: .ساعت (روشن) | .ساعت 3 (انتخاب فونت) | .ساعت فونت (لیست) | .غیرفعال ساعت
- ایموجی پریمیوم (بدون اکانت پریمیوم، از طریق ربات اینلاین): .ایموجی متن [کد ایموجی]
- کپی پروفایل: روی پیام یه کاربر ریپلای کن و بنویس .کپی  (اسم، بیو، عکس) | برگشت: .غیرفعال کپی

امنیت: سشن‌ها رمزنگاری‌شده ذخیره می‌شن (Fernet)، سلف هیچ‌وقت روی ربات روباهیو کلیک نمی‌کنه،
محدودیت سرعت داره و کاربر هر لحظه می‌تونه اتصال رو قطع کنه (خروج کامل از سشن).

متغیرهای Railway: SELF_API_ID و SELF_API_HASH (از my.telegram.org)، اختیاری: SELF_KEY.
"""
import asyncio
import base64
import hashlib
import json
import logging
import os
import re
import time
from datetime import datetime, timezone

from sqlalchemy import BigInteger, Boolean, Column, DateTime, String, Text
from telegram import InlineKeyboardButton as B, InlineKeyboardMarkup as KB,
KeyboardButton, ReplyKeyboardMarkup, ReplyKeyboardRemove
from telegram.ext import ApplicationHandlerStop, CallbackQueryHandler,
CommandHandler, MessageHandler, filters

from database import Base, get_session

logger = logging.getLogger(__name__)

MAX_SLOTS = 3
MAX_DELAY = 3600
MIN_AUTO = 5       # هر: حالت خودکار یعنی مقدار این => تایمر N گپ می‌گه توی خودش رو کلمه ثانیه
```

```
POS = {"l": "⬅️ چپ", "m": "⏺️ وسط", "r": "➡️ راست", "n": "⬜ خالی"}
API_ID = int(os.environ.get("SELF_API_ID", "0") or 0)
API_HASH = os.environ.get("SELF_API_HASH", "").strip()
BOT_TOKEN = os.environ.get("BOT_TOKEN", "").strip()


class SelfAccess(Base):
    __tablename__ = "self_access"
    user_id = Column(BigInteger, primary_key=True)
    enabled = Column(Boolean, default=True)
    paused = Column(Boolean, default=False)
    phone = Column(String, nullable=True)
    session_enc = Column(Text, nullable=True)
    slots = Column(Text, default="[]")
    created_at = Column(DateTime(timezone=True), default=lambda:
datetime.now(timezone.utc))


class SelfProfile(Base):
    """وضعیت ساعت و کپی پروفایل هر کاربر (JSON)."""
    __tablename__ = "self_profile"
    user_id = Column(BigInteger, primary_key=True)
    data = Column(Text, default="{}")


# ---------- ابزارها ----------
_FA = str.maketrans("۰۱۲۳۴۵۶۷۸۹٠١٢٣٤٥٦٧٨٩", "01234567890123456789")


def fa2en(s):
    return (s or "").translate(_FA)


def norm(s):
    return re.sub(r"[\u200c\u200d\ufe0f\s]+", " ", s or "").strip().lower()


def _fernet():
    from cryptography.fernet import Fernet
    raw = os.environ.get("SELF_KEY", "").strip() or ("rubahio-self:" + BOT_TOKEN)
    return Fernet(base64.urlsafe_b64encode(hashlib.sha256(raw.encode()).digest()))


def enc(s):
    return _fernet().encrypt(s.encode()).decode()


def dec(s):
    return _fernet().decrypt(s.encode()).decode()
```

```
def configured():
    return bool(API_ID and API_HASH)


def get_row(uid):
    s = get_session()
    try:
        r = s.get(SelfAccess, uid)
        if r:
            s.expunge(r)
        return r
    finally:
        s.close()


def save_row(uid, **kw):
    s = get_session()
    try:
        r = s.get(SelfAccess, uid)
        if not r:
            r = SelfAccess(user_id=uid, enabled=True, paused=False, slots="[]")
            s.add(r)
        for k, v in kw.items():
            setattr(r, k, v)
        s.commit()
    finally:
        s.close()


def slots_of(row):
    try:
        v = json.loads(row.slots or "[]")
        return [x for x in v if isinstance(x, dict)][:MAX_SLOTS]
    except Exception:
        return []


def parse_delay(text):
    m = re.match(r"^(\d{1,5})\s*(s|m|دقیقه|ثانیه)?$", fa2en(text).strip().lower())
    if not m:
        return None
    n = int(m.group(1)) * (60 if m.group(2) in ("m", "دقیقه") else 1)
    return n if 0 <= n <= MAX_DELAY else None


def fmt_delay(n):
    n = int(n or 0)
    return f"{n // 60} دقیقه" if n >= 60 and n % 60 == 0 else f"{n} ثانیه"
```

```
# ---------- هسته‌ی سلف (Telethon) ----------
_clients = {}
_slots = {}
_paused = {}
_hist = {}
_last = {}
_login = {}
_bot_id = [0]
_capture = {}
_botref = [None]
_own = {}   # (uid, chat, کلمه) -> زمان؛ پیام‌هایی که خود سلف می‌فرسته و نباید دوباره تریگر بشن


def _load_runtime(uid, row):
    _slots[uid] = slots_of(row)
    _paused[uid] = bool(row.paused)
    _sync_auto(uid)


async def stop_client(uid, logout=False, restore=False):
    c = _clients.pop(uid, None)
    _cancel_auto(uid)
    _cancel_clock(uid)
    _slots.pop(uid, None)
    if c and restore:
        try:
            await _reset_profile(uid, c)
        except Exception as e:  # noqa: BLE001
            logger.info("self reset profile %s: %s", uid, e)
    if c:
        try:
            if logout:
                await c.log_out()
            else:
                await c.disconnect()
        except Exception:
            try:
                await c.disconnect()
            except Exception:
                pass


async def start_client(uid):
    if not configured():
        return False
    from telethon import TelegramClient, events
    from telethon.sessions import StringSession
    row = get_row(uid)
    if not (row and row.enabled and row.session_enc):
```

```
        return False
    await stop_client(uid)
    try:
        client = TelegramClient(StringSession(dec(row.session_enc)), API_ID,
API_HASH)
        await client.connect()
        if not await client.is_user_authorized():
            save_row(uid, session_enc=None)
            await client.disconnect()
            return False
    except Exception as e:  # noqa: BLE001
        logger.warning("self start failed %s: %s", uid, e)
        return False
    _load_runtime(uid, row)

    async def _h(ev, _uid=uid):
        await _on_out(_uid, ev)
    client.add_event_handler(_h, events.NewMessage(outgoing=True))
    _clients[uid] = client
    _sync_auto(uid)
    _sync_clock(uid)
    return True


async def _on_out(uid, ev):
    cap = _capture.get(uid)
    if cap:
        if time.time() - cap["ts"] > 300:
            _capture.pop(uid, None)
        elif ev.chat_id == cap["chat"] and (ev.raw_text or "").strip():
            _capture.pop(uid, None)
            sl = _get_slots(uid)
            sl[cap["slot"]]["word"] = (ev.raw_text or "").strip()[:40]
            sl[cap["slot"]].setdefault("delay", 0)
            _put_slots(uid, sl)
            try:
                tx, kb = _slot_panel(uid, cap["slot"])
                await _botref[0].send_message(uid, f"✅ کلمه خودکار ثبت شد:
«{sl[cap['slot']]['word']}»\n\n" + tx,
                                              reply_markup=kb, parse_mode="HTML")
            except Exception as e:  # noqa: BLE001
                logger.info("self capture notify failed: %s", e)
            return
    if await _handle_dot(uid, ev):
        return
    if _paused.get(uid):
        return
    txt = norm(ev.raw_text or "")
    if not txt:
        return
```

```
    if _own.pop((uid, ev.chat_id, txt), 0) > time.time():
        return   # پیامی که خود سلف فرستاده؛ دوباره تریگر نشه
    for i, s in enumerate(_slots.get(uid, [])):
        if s and s.get("word") and s.get("chat") and ev.chat_id == s["chat"] and
ev.chat_id != _bot_id[0] and norm(s["word"]) == txt and int(s.get("delay", 0) or 0) <
MIN_AUTO:
            now = time.time()
            if now - _last.get((uid, i), 0) < 2:
                continue
            h = [t for t in _hist.get(uid, []) if now - t < 60]
            if len(h) >= 20:
                return
            h.append(now)
            _hist[uid] = h
            _last[(uid, i)] = now
            asyncio.create_task(_fire(uid, ev, s))


def _pick_button(msg, pos):
    rows = [r for r in (msg.buttons or []) if r]
    if not rows or pos == "n":
        return None
    row = rows[0]
    if pos == "l":
        return row[0]
    if pos == "r":
        return row[-1]
    if len(row) == 1:
        return row[0]
    return row[len(row) // 2] if len(row) % 2 == 1 else None


async def _usable(m):
    """پیام دکمه‌دار از یک ربات؛ هیچ‌وقت ربات روباهیو."""
    if not m.buttons or m.sender_id == _bot_id[0]:
        return False
    snd = await m.get_sender()
    return bool(snd and getattr(snd, "bot", False))


async def _fire(uid, ev, s):
    try:
        await asyncio.sleep(max(0, int(s.get("delay", 0))))
        client = _clients.get(uid)
        if not client or _paused.get(uid) or ev.chat_id == _bot_id[0]:
            return
        pos = s.get("pos", "n")
        # سر تایمر کلمه رو توی همون گپ می‌گه (ریپلای هم باشه، ریپلای می‌کنه)
        word = (s.get("word") or "").strip()
        if word:
```

```
            _own[(uid, ev.chat_id, norm(word))] = time.time() + 15
            await client.send_message(ev.chat_id, word, reply_to=ev.reply_to_msg_id
if ev.is_reply else None)
        if pos == "n":
            return
        target = None
        if ev.is_reply:
            m = await ev.get_reply_message()
            if m and await _usable(m):
                target = m
        if not target:
            async for m in client.iter_messages(ev.chat_id, limit=15):
                if m.buttons and await _usable(m):
                    target = m
                    break
        btn = _pick_button(target, pos) if target else None
        if btn:
            await btn.click()
    except Exception as e:  # noqa: BLE001
        logger.info("self fire failed %s: %s", uid, e)


# ---------- حالت خودکار: هر N ثانیه کلمه رو خودش توی گپ می‌گه و دکمه رو می‌زنه ----------
_auto = {}   # (uid, اسلات) -> (امضا، تسک)


def _auto_sig(s):
    return (s.get("word"), s.get("chat"), int(s.get("delay", 0) or 0), s.get("pos",
"n"))


def _cancel_auto(uid):
    for k in [k for k in _auto if k[0] == uid]:
        try:
            _auto.pop(k)[1].cancel()
        except Exception:
            pass


def _sync_auto(uid):
    """ری‌استارت همون اسلات) = تغییر تنظیمات) تسک‌های خودکار رو با تنظیمات فعلی هماهنگ می‌کنه."""
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        return
    sl = _slots.get(uid, []) if uid in _clients else []
    want = {}
    for i, s in enumerate(sl):
        if s and s.get("word") and s.get("chat") and int(s.get("delay", 0) or 0) >=
MIN_AUTO:
```

```
            want[(uid, i)] = _auto_sig(s)
    for k in [k for k in _auto if k[0] == uid and (k not in want or _auto[k][0] !=
want[k])]:
        _auto.pop(k)[1].cancel()
    for k, sig in want.items():
        if k not in _auto:
            _auto[k] = (sig, loop.create_task(_auto_loop(uid, k[1], sig)))


async def _auto_loop(uid, i, sig):
    delay = sig[2]
    try:
        while True:
            await asyncio.sleep(delay)
            client = _clients.get(uid)
            sl = _slots.get(uid, [])
            cur = sl[i] if i < len(sl) else None
            if not client or not cur or _auto_sig(cur) != sig:
                return
            if _paused.get(uid):
                continue
            now = time.time()
            h = [t for t in _hist.get(uid, []) if now - t < 60]
            if len(h) >= 20:
                continue
            h.append(now)
            _hist[uid] = h
            try:
                await _auto_fire(uid, client, cur)
            except asyncio.CancelledError:
                raise
            except Exception as e:  # noqa: BLE001
                logger.info("self auto fire %s/%s: %s", uid, i, e)
                wait = getattr(e, "seconds", None)
                if isinstance(wait, int) and wait > 0:
                    await asyncio.sleep(min(wait, 300))
    except asyncio.CancelledError:
        raise


async def _auto_fire(uid, client, s):
    chat = s["chat"]
    word = (s.get("word") or "").strip()
    if not word or chat == _bot_id[0]:
        return
    _own[(uid, chat, norm(word))] = time.time() + 15
    sent = await client.send_message(chat, word)
    pos = s.get("pos", "n")
    if pos == "n":
        return
```

```
    target = None
    for _ in range(4):                      # جواب ربات رو چند ثانیه صبر می‌کنه
        await asyncio.sleep(1.5)
        async for m in client.iter_messages(chat, limit=10):
            if m.id <= sent.id:
                break
            if m.buttons and await _usable(m):
                target = m
                break
        if target:
            break
    if not target:                          # جواب تازه نیومد؛ آخرین پیام دکمه‌دار رباتا
        async for m in client.iter_messages(chat, limit=15):
            if m.buttons and await _usable(m):
                target = m
                break
    btn = _pick_button(target, pos) if target else None
    if btn:
        await btn.click()


async def start_all(app):
    _bot_id[0] = app.bot.id
    _botref[0] = app.bot
    if not configured():
        logger.info("self: SELF_API_ID/SELF_API_HASH تنظیم نشده؛ سلف غیرفعال است")
        return
    s = get_session()
    try:
        ids = [r.user_id for r in
s.query(SelfAccess).filter(SelfAccess.enabled.is_(True),
SelfAccess.session_enc.isnot(None)).all()]
    finally:
        s.close()

    async def _go():
        for uid in ids:
            try:
                await start_client(uid)
            except Exception as e:  # noqa: BLE001
                logger.warning("self autostart %s: %s", uid, e)
            await asyncio.sleep(1.5)
    asyncio.create_task(_go())


async def _dialogs(client):
    out = []
    try:
        async for d in client.iter_dialogs(limit=300):
            if d.id == _bot_id[0]:
```

```
                continue
            if d.is_group:
                out.append((d.id, d.name or str(d.id), True))
            elif d.is_user and getattr(d.entity, "bot", False):
                out.append((d.id, d.name or str(d.id), False))
    except Exception as e:  # noqa: BLE001
        logger.info("dialogs failed: %s", e)
    return out


# ---------- ساعت زنده کنار اسم + کپی پروفایل ----------
# هر فونت: (اسم، ۱۰ رقم از 0 تا 9، علامت دونقطه)
CLOCK_FONTS = [
    ("0123456789" ,"عادی", ":"),
    ("𝟬𝟭𝟮𝟯𝟰𝟱𝟲𝟳𝟴𝟵" ,"بولد", ":"),
    ("𝟘𝟙𝟚𝟛𝟜𝟝𝟞𝟟𝟠𝟡" ,"دابل", ":"),
    ("0123456789" ,"مونو", ":"),
    ("𝟬𝟭𝟮𝟯𝟰𝟱𝟲𝟳𝟴𝟵" ,"سنس بولد", ":"),
    ("0123456789" ,"سنس", ":"),
    ("دایره‌ای", "⓪①②③④⑤⑥⑦⑧⑨", ":"),
    ("فول‌ویدth" ,"0 1 2 3 4 5 6 7 8 9", ": "),
    ("فارسی" ,"۰۱۲۳۴۵۶۷۸۹", ":"),
    ("بالانویس" ,"0 1 2 3 56789", ":"),
]
CLOCK_TZ = os.environ.get("SELF_TZ", "Asia/Tehran").strip() or "Asia/Tehran"
_clock_tasks = {}


def _tz():
    try:
        from zoneinfo import ZoneInfo
        return ZoneInfo(CLOCK_TZ)
    except Exception:
        from datetime import timedelta
        return timezone(timedelta(hours=3, minutes=30))   # ایران (بدون ساعت تابستانی)


def fmt_clock(font=1, now=None):
    font = font if isinstance(font, int) and 1 <= font <= len(CLOCK_FONTS) else 1
    _, digits, colon = CLOCK_FONTS[font - 1]
    t = (now or datetime.now(_tz())).strftime("%H:%M")
    return "".join(colon if ch == ":" else digits[int(ch)] for ch in t)


def _u16(s):
    return len(s.encode("utf-16-le")) // 2


def build_first_name(base_first, clock_txt=None, limit=64):
    """اسم + ساعت؛ اگه اسم بلند بود اسم کوتاه می‌شه تا ساعت جا بشه."""
```

```
    base_first = (base_first or "").strip()
    if not clock_txt:
        return base_first[:limit] or "."
    room = limit - _u16(clock_txt) - 1
    if room <= 0 or not base_first:
        return clock_txt
    while base_first and _u16(base_first) > room:
        base_first = base_first[:-1]
    return (base_first.rstrip() + " " + clock_txt) if base_first else clock_txt


def _pget(uid):
    s = get_session()
    try:
        r = s.get(SelfProfile, uid)
        return json.loads(r.data or "{}") if r else {}
    except Exception:
        return {}
    finally:
        s.close()


def _pput(uid, d):
    s = get_session()
    try:
        r = s.get(SelfProfile, uid)
        if not r:
            r = SelfProfile(user_id=uid)
            s.add(r)
        r.data = json.dumps(d, ensure_ascii=False)
        s.commit()
    finally:
        s.close()


async def _ensure_orig(uid, client):
    """پروفایل اصلی کاربر (قبل از ساعت/کپی) یک‌بار ذخیره می‌شه تا برگردوندنش ممکن باشه."""
    d = _pget(uid)
    if "orig_first" in d:
        return d
    from telethon.tl.functions.users import GetFullUserRequest
    me = await client.get_me()
    full = await client(GetFullUserRequest(me))
    d["orig_first"] = me.first_name or ""
    d["orig_last"] = me.last_name or ""
    d["orig_about"] = getattr(full.full_user, "about", None) or ""
    _pput(uid, d)
    return d
```

```
def _maybe_clear(uid):
    d = _pget(uid)
    if not d.get("clock_on") and not d.get("copy_on"):
        _pput(uid, {"font": d.get("font", 1)})


async def _apply_name(uid, client, about=None):
    from telethon.tl.functions.account import UpdateProfileRequest
    d = _pget(uid)
    if d.get("copy_on"):
        bf, bl = d.get("copy_first", ""), d.get("copy_last", "")
    else:
        bf, bl = d.get("orig_first", ""), d.get("orig_last", "")
    clk = fmt_clock(d.get("font", 1)) if d.get("clock_on") else None
    first = build_first_name(bf, clk)
    await client(UpdateProfileRequest(first_name=first, last_name=(bl or "")[:64],
about=about))
    d["last_clock"] = clk
    _pput(uid, d)


def _cancel_clock(uid):
    t = _clock_tasks.pop(uid, None)
    if t:
        try:
            t.cancel()
        except Exception:
            pass


def _sync_clock(uid):
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        return
    t = _clock_tasks.get(uid)
    if t and not t.done():
        return
    if uid in _clients and _pget(uid).get("clock_on"):
        _clock_tasks[uid] = loop.create_task(_clock_loop(uid))


async def _clock_loop(uid):
    from telethon import errors
    last = None
    try:
        while True:
            client = _clients.get(uid)
            d = _pget(uid)
            if not client or not d.get("clock_on"):
```

```
                return
            cur = fmt_clock(d.get("font", 1))
            wait = 0
            if cur != last:
                try:
                    await _apply_name(uid, client)
                    last = cur
                except errors.FloodWaitError as e:
                    wait = min(int(getattr(e, "seconds", 60)) + 2, 3600)
                except Exception as e:  # noqa: BLE001
                    logger.info("self clock %s: %s", uid, e)
                    wait = 30
            if not wait:
                wait = 60 - (time.time() % 60) + 0.7
            await asyncio.sleep(wait)
    except asyncio.CancelledError:
        raise
    finally:
        if _clock_tasks.get(uid) is asyncio.current_task():
            _clock_tasks.pop(uid, None)


def clock_fonts_text(uid):
    d = _pget(uid)
    cur = d.get("font", 1)
    lines = [f"🕒 ساعت: {'✅ روشن' if d.get('clock_on') else '❌ خاموش'}   فونت فعلی: {cur}", ""]
    for i, (name, _, _) in enumerate(CLOCK_FONTS, 1):
        lines.append(f"{i}) {fmt_clock(i)}   {name}" + ("  ✔️" if i == cur else ""))
    lines += ["", "غیرفعال ساعت. :خاموش   |   3 ساعت. :انتخاب"]
    return "\n".join(lines)


async def clock_set(uid, on=None, font=None):
    client = _clients.get(uid)
    if not client:
        return "❌ اول اکانتت رو وصل کن (/self)."
    d = _pget(uid)
    if font is not None:
        d["font"] = font
    try:
        if on is False:
            if not d.get("clock_on"):
                return "🕒 ساعت از قبل خاموشه."
            d["clock_on"] = False
            _pput(uid, d)
            _cancel_clock(uid)
            await _apply_name(uid, client)
            _maybe_clear(uid)
            return "✅ ساعت خاموش شد و اسمت برگشت."
```

```
        d = await _ensure_orig(uid, client) if not d.get("clock_on") else d
        if font is not None:
            d["font"] = font
        d["clock_on"] = True
        _pput(uid, d)
        await _apply_name(uid, client)
        _sync_clock(uid)
        return f"✅ ساعت زنده روشن شد: {fmt_clock(d.get('font', 1))}  (فونت
{d.get('font', 1)})\nتغییر فونت: .ساعت 2  |  لیست: .ساعت فونت"
    except Exception as e:  # noqa: BLE001
        logger.info("self clock_set %s: %s", uid, e)
        return "⚠️ خطا: " + str(e)[:120]


async def _delete_copy_photo(client, d):
    pid = d.pop("copy_photo", None)
    if not pid:
        return
    from telethon import utils
    from telethon.tl.functions.photos import DeletePhotosRequest
    async for ph in client.iter_profile_photos("me", limit=10):
        if ph.id == pid:
            await client(DeletePhotosRequest([utils.get_input_photo(ph)]))
            break


async def copy_on(uid, ev):
    client = _clients.get(uid)
    if not client:
        return "❌ اول اکانتت رو وصل کن (/self)."
    if not ev.is_reply:
        return "↩️ روی پیام اون کاربر ریپلای کن و بنویس: .کپی"
    try:
        import io
        from telethon.tl.types import User as TUser
        from telethon.tl.functions.users import GetFullUserRequest
        from telethon.tl.functions.photos import UploadProfilePhotoRequest
        msg = await ev.get_reply_message()
        target = await msg.get_sender() if msg else None
        if not isinstance(target, TUser):
            return "❌ فقط روی پیام یه کاربر می‌شه کپی کرد (نه کانال/ادمین ناشناس)."
        me = await client.get_me()
        if target.id == me.id:
            return "😅 این خودتی!"
        d = await _ensure_orig(uid, client)
        full = await client(GetFullUserRequest(target))
        about = getattr(full.full_user, "about", None) or ""
        about = about[: (140 if getattr(me, "premium", False) else 70)]
        first = (target.first_name or "").strip() or (target.username or "") or "."
        last = (target.last_name or "").strip()
```

```
        photo_bytes = None
        try:
            photo_bytes = await client.download_profile_photo(target, file=bytes)
        except Exception as e:  # noqa: BLE001
            logger.info("self copy photo dl: %s", e)
        await _delete_copy_photo(client, d)    # اگه قبلاً کپی دیگه‌ای فعال بود، عکسش پاک بشه
        new_pid = None
        if photo_bytes:
            bio = io.BytesIO(photo_bytes)
            bio.name = "photo.jpg"
            up = await client.upload_file(bio)
            res = await client(UploadProfilePhotoRequest(file=up))
            new_pid = getattr(getattr(res, "photo", None), "id", None)
        d["copy_on"] = True
        d["copy_first"], d["copy_last"] = first[:64], last[:64]
        d["copy_photo"] = new_pid
        d["copy_about"] = about
        _pput(uid, d)
        try:
            await _apply_name(uid, client, about=about)
        except Exception:
            await _apply_name(uid, client, about=about[:70])
        _sync_clock(uid)
        extra = "" if photo_bytes else "\n(عکس نداشت یا مخفیه؛ عکس خودت دست نخورد.)"
        return f"✅ پروفایل «{first}» کپی شد (اسم، بیو" + ("، عکس" if photo_bytes else "")
+ ")." + extra + "\nبرگشت: /غیرفعال کپی"
    except Exception as e:  # noqa: BLE001
        logger.info("self copy %s: %s", uid, e)
        return "⚠️ کپی نشد: " + str(e)[:120]


async def copy_off(uid):
    client = _clients.get(uid)
    if not client:
        return "❌ اول اکانتت رو وصل کن (/self)."
    d = _pget(uid)
    if not d.get("copy_on"):
        return "ℹ️ کپی فعال نیست."
    try:
        await _delete_copy_photo(client, d)
        for k in ("copy_on", "copy_first", "copy_last", "copy_about"):
            d.pop(k, None)
        _pput(uid, d)
        await _apply_name(uid, client, about=d.get("orig_about", ""))
        _maybe_clear(uid)
        return "✅ کپی غیرفعال شد؛ اسم، بیو و عکس خودت برگشت."
    except Exception as e:  # noqa: BLE001
        logger.info("self copy_off %s: %s", uid, e)
        return "⚠️ خطا: " + str(e)[:120]
```

```
async def _reset_profile(uid, client):
    """موقع قطع اتصال/غیرفعال‌سازی: ساعت و کپی برداشته می‌شن و پروفایل اصلی برمی‌گرده."""
    d = _pget(uid)
    if not d.get("clock_on") and not d.get("copy_on"):
        return
    had_copy = bool(d.get("copy_on"))
    if had_copy:
        await _delete_copy_photo(client, d)
    for k in ("copy_on", "copy_first", "copy_last", "copy_about"):
        d.pop(k, None)
    d["clock_on"] = False
    _pput(uid, d)
    await _apply_name(uid, client, about=d.get("orig_about", "") if had_copy else
None)
    _maybe_clear(uid)


def _clock_panel(uid):
    d = _pget(uid)
    on = bool(d.get("clock_on"))
    cur = d.get("font", 1)
    tx = (f"🕒 <b>ساعت زنده</b>\n\nوضعیت: {'✅ روشن' if on else '❌ خاموش'}\n"
          f"نمونه: {fmt_clock(cur)}\n\nهر دقیقه به‌روز می‌شه. ساعت کنار اسم اکانتت (به وقت ایران)
فونت دلخواهت رو انتخاب کن:\n"
          "⚠️ تا ساعت روشنه اسمت رو دستی عوض نکن؛ اول ساعت رو خاموش کن.")
    kb = [[B("⏹ خاموش کردن" if on else "▶️ روشن کردن", callback_data="self:clkt")]]
    row = []
    for i in range(1, len(CLOCK_FONTS) + 1):
        row.append(B(f"{'✔️ ' if i == cur else ''}{fmt_clock(i)}",
callback_data=f"self:clkf:{i}"))
        if len(row) == 2:
            kb.append(row)
            row = []
    if row:
        kb.append(row)
    kb.append([B("🔙 بازگشت", callback_data="self:home")])
    return tx, KB(kb)


EMOJI_BOT = (os.environ.get("SELF_EMOJI_BOT", "AS_EmojiBot").strip() or
"AS_EmojiBot").lstrip("@")


async def send_premium(client, chat, text, reply_to=None):
    """ایموجی پریمیوم از راه ربات اینلاین: همون کاری که کاربر دستی می‌کنه (@AS_EmojiBot [کد] متن)."""
    res = await client.inline_query(EMOJI_BOT, text, entity=chat)
    if not res:
        return False
    await res[0].click(chat, reply_to=reply_to)
```

```
    return True


def _u16(t):
    return len(t.encode("utf-16-le")) // 2


async def emoji_send(uid, ev, arg):
    """(بدون ربات واسطه) ایموجی پریمیوم مستقیم از خود اکانت.
    دو حالت: ۱) ایموجی پریمیوم رو از کیبوردت توی متن بذار ۲) {آیدی عددی ایموجی} بنویس
    اکانت باید پریمیوم باشه."""
    import copy
    from telethon.tl.types import MessageEntityCustomEmoji
    client = _clients.get(uid)
    if not client:
        return "❌ اول اکانتت رو وصل کن (/self)."
    if not arg:
        return "✨ (ایموجی پریمیوم)😎 ایموجی سلام. :مثل ایموجی سلام\nیا با آیدی: .ایموجی سلام
{5368324170671202286}"
    try:
        me = await client.get_me()
        if not getattr(me, "premium", False):
            return "❌ اکانتی که وصل کردی پریمیوم نیست؛ ایموجی پریمیوم فقط از اکانت پریمیوم ارسال
می‌شه."
        raw = ev.raw_text or ""
        end = len(raw.rstrip())
        start = end - len(arg)
        start16 = _u16(raw[:start])
        ents = []
        for e in (ev.message.entities or []):
            if e.offset >= start16:
                e2 = copy.copy(e)
                e2.offset -= start16
                ents.append(e2)
        text = arg
        # جایگزینی {آیدی} با ایموجی پریمیوم
        while True:
            m = re.search(r"\{(\d{6,})\}", text)
            if not m:
                break
            pos16 = _u16(text[:m.start()])
            old16 = _u16(m.group(0))
            for e in ents:
                if e.offset >= pos16 + old16:
                    e.offset -= old16 - 1
            ents.append(MessageEntityCustomEmoji(offset=pos16, length=1,
document_id=int(m.group(1))))
            text = text[:m.start()] + "⭐" + text[m.end():]
        if not any(isinstance(e, MessageEntityCustomEmoji) for e in ents):
            return "❌ توی متن ایموجی پریمیوم پیدا نشد؛ از کیبوردت بذار یا {آیدی} بنویس."
```

```
        ents.sort(key=lambda e: e.offset)
        await client.send_message(ev.chat_id, text, formatting_entities=ents,
parse_mode=None,
                                  reply_to=ev.reply_to_msg_id if ev.is_reply else
None)
    except Exception as e:  # noqa: BLE001
        logger.info("self emoji %s: %s", uid, e)
        return "⚠️ ارسال نشد: " + str(e)[:120]
    try:
        await ev.delete()      # پیام دستور پاک می‌شه و فقط پیام ایموجی می‌مونه
    except Exception:
        pass
    return None


# ---------- دستورهای نقطه‌ای داخل چت (.پنل / .کلمه / .زمان / .دکمه / .گپ) ----------
# این دستورها رو می‌نویسه (Saved Messages هر چتی؛ بهترین جا) کاربرِ فعال‌شده از خود اکانتش.
#   .پنل → پنل تنظیمات بالا میاد.
#   .کلمه فلان → کلمه‌ی اسلات فعلی (اگه عدد اسلات بذاری: .کلمه 2 فلان).
#   .زمان فلان → تایمر: 5 یا 10. یا 2 دقیقه sزمان فلان
#   .دکمه فلان → چپ | وسط | راست | خالی.
#   .گپ → همین گپی که توش می‌نویسی؛ یا .گپ اسم گروه.
# بعد از تنظیم (کلمه + گپ + دکمه) سلف خودش کار می‌کنه.
_CMD = {"پنل": "panel", "panel": "panel", "منو": "panel",
        "کلمه": "word", "word": "word",
        "زمان": "time", "تایمر": "time", "time": "time",
        "دکمه": "btn", "button": "btn",
        "گپ": "chat", "گروه": "chat", "chat": "chat",
        "ساعت": "clock", "clock": "clock",
        "کپی": "copy", "copy": "copy",
        "ایموجی": "emoji", "emoji": "emoji", "پرمیوم": "emoji", "پریمیوم": "emoji",
"premium": "emoji",
        "غیرفعال": "off", "خاموش": "off", "off": "off", "غیر": "off"}
_POSW = {"چپ": "l", "left": "l", "l": "l", "وسط": "m", "میانه": "m", "mid": "m",
"middle": "m", "m": "m",
         "راست": "r", "right": "r", "r": "r", "خالی": "n", "هیچ": "n", "none": "n",
"n": "n"}
_AR2FA = str.maketrans("یک" ,"يك")
_cur = {}          # uid -> (دستورهای بدون شماره روی همین اعمال می‌شن) اسلاتی که آخر دست زدیم
_cmd_hist = {}


def parse_dot(raw):
    """.کلمه سلام' → ('word', 'سلام'). اگه دستور شناخته‌شده نبود None."""
    t = (raw or "").strip().translate(_AR2FA)
    if not t.startswith("."):
        return None
    m = re.match(r"^\.\s*(\S+)\s*(.*)$", t, re.S)
    if not m:
        return None
```

```
    cmd = _CMD.get(m.group(1).replace("\u200c", "").lower())
    return (cmd, m.group(2).strip()) if cmd else None


def split_slot(arg):
    """'2 بدون عدد اسلات .('سلام' ,1) → 'سلام → (None, arg)."""
    m = re.match(r"^([1-3\u06f1-\u06f3])\s+(\S.*)$", arg or "", re.S)
    if m:
        return int(fa2en(m.group(1))) - 1, m.group(2).strip()
    return None, (arg or "").strip()


def _slot_line(i, s):
    if not s or not s.get("word"):
        return f"{i + 1}  ➕ خالی"
    parts = [f"{i + 1}  «{s['word']}»", f"💬 {(s.get('title') or 'گپ؟')[:22]}",
             (f"🔁 هر {fmt_delay(s.get('delay', 0))}" if int(s.get('delay', 0) or 0)
>= MIN_AUTO else "🔁 خاموش"),
             f"🎯 {POS[s.get('pos', 'n')]}"]
    return " | ".join(parts) + ("  ✅" if s.get("chat") else "  ⚠️ گپ تنظیم نشده")


def _missing(s):
    miss = []
    if not s.get("word"):
        miss.append("کلمه فلان.")
    if not s.get("chat"):
        miss.append("گپ.")
    if int(s.get("delay", 0) or 0) < MIN_AUTO:
        miss.append("زمان 30 (هر چند ثانیه خودکار بگه؛ حداقل 5.)")
    return miss


def _ready_text(s):
    miss = _missing(s)
    if not miss:
        tail = (f" و بعدش دکمه‌ی {POS[s.get('pos')]} رو می‌زنه." if s.get("pos", "n") !=
"n"
                else " (دکمه‌ای زده نمی‌شه؛ اگه می‌خوای: .دکمه چپ|وسط|راست).")
        return (f"🚀 هر فعاله! {fmt_delay(s.get('delay', 0))} خودش «{s['word']}» رو توی
«{s.get('title') or 'گپ'}» می‌گه" + tail)
    return "📌 مونده: " + " | ".join(miss)


def _panel_text(uid):
    row = get_row(uid)
    sl = slots_of(row) if row else []
    on = uid in _clients
    lines = ["🤖 سلف روبی", "",
             f"🔌 اتصال: {'✅ وصل' if on else '❌ قطع'}   ⚙️ وضعیت: {'⏸️ متوقف' if row
```

```
and row.paused else '▶️ فعال'}", ""]
    for i in range(MAX_SLOTS):
        lines.append(_slot_line(i, sl[i] if i < len(sl) else {}))
    lines += ["",
              "(روی اسلات فعلی؛ برای اسلات دیگه عدد بذار: .کلمه 2 سلام):دستورها",
              "کلمه فلان — کلمه‌ای که می‌فرستی.",
              "زمان فلان — هر چند ثانیه خودکار بگه (حداقل 5؛ مثل 30 یا 2 دقیقه).",
              "دکمه فلان — چپ | وسط | راست | خالی.",
              "گپ — همین گپ؛ یا .گپ اسم گروه.",
              "ساعت — ساعت زنده کنار اسم | .ساعت 3 فونت | .غیرفعال ساعت.",
              "ایموجی متن [کد] — ارسال ایموجی پریمیوم با ربات @AS_EmojiBot.",
              "کپی (ریپلای روی کاربر) — کپی اسم/بیو/عکس | .غیرفعال کپی.",
              "", "بعد از تنظیم کلمه + گپ + زمان، سلف خودش هر چند ثانیه کلمه رو می‌گه و دکمه رو می‌زنه؛
لازم نیست خودت چیزی بفرستی."]
    return "\n".join(lines)


async def _del_later(m, sec):
    try:
        await asyncio.sleep(sec)
        await m.delete()
    except Exception:
        pass


async def _say(uid, ev, text, keep=False):
    """جواب رو همون‌جا (با ادیت پیام دستور) می‌ده و بعد چند ثانیه پاکش می‌کنه."""
    try:
        m = await ev.edit(text, parse_mode=None)
    except Exception:
        try:
            await _botref[0].send_message(uid, text)
        except Exception:
            pass
        return
    if not keep:
        asyncio.create_task(_del_later(m, 10))


def _pick_slot(uid, sl, explicit, word=None):
    if explicit is not None:
        return explicit
    if word:
        w = norm(word)
        for i, s in enumerate(sl):
            if s.get("word") and norm(s["word"]) == w:
                return i
        for i, s in enumerate(sl):
            if not s.get("word"):
                return i
```

```
    return min(max(_cur.get(uid, 0), 0), MAX_SLOTS - 1)


async def _find_chat(client, name):
    q = norm(name)
    ds = await _dialogs(client)
    exact = [d for d in ds if norm(d[1]) == q]
    if len(exact) == 1:
        return [exact[0]]
    return [d for d in ds if q in norm(d[1])]


async def _handle_dot(uid, ev):
    raw = ev.raw_text or ""
    if not raw.lstrip().startswith("."):
        return False
    pc = parse_dot(raw)
    if not pc:
        return False
    cmd, arg = pc
    now = time.time()
    h = [t for t in _cmd_hist.get(uid, []) if now - t < 60]
    if len(h) >= 40:
        return True
    h.append(now)
    _cmd_hist[uid] = h
    try:
        if cmd == "panel":
            if ev.chat_id == uid:      # Saved Messages: پنل همون‌جا بالا میاد
                await ev.edit(_panel_text(uid), parse_mode=None)
            else:                      # گپ عمومی: پیام دستور پاک می‌شه و پنل می‌ره توی چت ربات
                try:
                    await ev.delete()
                except Exception:
                    pass
                tx, kb = _panel(uid)
                await _botref[0].send_message(uid, _panel_text(uid), reply_markup=kb)
            return True
        if cmd == "off":
            what = norm(re.sub(r"^فعال\s*", "", arg or ""))
            if what in ("کپی", "copy"):
                await _say(uid, ev, await copy_off(uid))
            elif what in ("ساعت", "clock"):
                await _say(uid, ev, await clock_set(uid, on=False))
            else:
                await _say(uid, ev, "❓ مثل: .غیرفعال کپی  یا  .غیرفعال ساعت")
            return True
        if cmd == "emoji":
            msg_out = await emoji_send(uid, ev, arg)
            if msg_out:
```

```
                await _say(uid, ev, msg_out)
            return True
        if cmd == "copy":
            if norm(arg) in ("خاموش", "off", "غیرفعال"):
                await _say(uid, ev, await copy_off(uid))
            else:
                await _say(uid, ev, await copy_on(uid, ev))
            return True
        if cmd == "clock":
            a = norm(arg)
            if a in ("خاموش", "off", "غیرفعال"):
                await _say(uid, ev, await clock_set(uid, on=False))
            elif a in ("فونتها" ,"فونت ها" ,"فونت", "fonts", "font", "لیست"):
                await _say(uid, ev, clock_fonts_text(uid), keep=True)
            elif fa2en(a).isdigit():
                n = int(fa2en(a))
                if 1 <= n <= len(CLOCK_FONTS):
                    await _say(uid, ev, await clock_set(uid, on=True, font=n))
                else:
                    await _say(uid, ev, clock_fonts_text(uid), keep=True)
            elif a == "":
                d = _pget(uid)
                if d.get("clock_on"):
                    await _say(uid, ev, clock_fonts_text(uid), keep=True)
                else:
                    await _say(uid, ev, await clock_set(uid, on=True))
            else:
                await _say(uid, ev, "🕒 ساعت. | ساعت فونت. | ساعت 3. | مثل: ساعت. غیرفعال ساعت.")
            return True
        explicit, rest = split_slot(arg)
        sl = _get_slots(uid)
        if cmd == "word":
            if not rest:
                await _say(uid, ev, "✏️ کلمه سلام. | مثل: کلمه سلام 2 (یا اسلات مشخص)")
                return True
            i = _pick_slot(uid, sl, explicit, rest)
            sl[i]["word"] = rest[:40]
            sl[i].setdefault("delay", 0)
            _put_slots(uid, sl)
            _cur[uid] = i
            await _say(uid, ev, f"✅ کلمه‌ی اسلات {i + 1}: «{sl[i]['word']}»\n" +
_ready_text(sl[i]))
        elif cmd == "time":
            d = parse_delay(rest)
            if d is None or 0 < d < MIN_AUTO:
                await _say(uid, ev, "⏱ زمان. 2 | یا زمان. 30 | هر چند ثانیه خودکار بگه؟ مثل: زمان. 0 = خودکار خاموش\n(حداقل 5 ثانیه، حداکثر ۶۰ دقیقه؛ دقیقه)")
                return True
            i = _pick_slot(uid, sl, explicit)
```

```
            sl[i]["delay"] = d
            _put_slots(uid, sl)
            _cur[uid] = i
            await _say(uid, ev, (f"✅ زمان اسلات {i + 1}: هر {fmt_delay(d)}\n" if d else
f"✅ حالت خودکار اسلات {i + 1} خاموش شد.\n") + _ready_text(sl[i]))
        elif cmd == "btn":
            pos = _POSW.get(norm(rest))
            if not pos:
                await _say(uid, ev, "🎯 مثل: .دکمه چپ یا .دکمه وسط یا .دکمه راست
یا .دکمه خالی")
                return True
            i = _pick_slot(uid, sl, explicit)
            sl[i]["pos"] = pos
            _put_slots(uid, sl)
            _cur[uid] = i
            await _say(uid, ev, f"✅ دکمه‌ی اسلات {i + 1}: {POS[pos]}\n" +
_ready_text(sl[i]))
        elif cmd == "chat":
            client = _clients.get(uid)
            slot_only = re.fullmatch(r"[1-3\u06f1-\u06f3]", arg or "")
            if slot_only:
                explicit, rest = int(fa2en(arg)) - 1, ""
            if not rest:      # همین گپ
                chat = await ev.get_chat()
                is_bot = bool(ev.is_private and getattr(chat, "bot", False))
                if ev.chat_id in (uid, _bot_id[0]) or not (ev.is_group or is_bot):
                    await _say(uid, ev, "💬 این دستور رو داخل گپِ هدف (گروه یا چت ربات) بنویس،
یا بنویس: .گپ اسم گروه")
                    return True
                cid = ev.chat_id
                title = getattr(chat, "title", None) or getattr(chat, "first_name",
None) or str(cid)
            else:
                found = await _find_chat(client, rest) if client else []
                if not found:
                    await _say(uid, ev, f"❌ گپی با اسم «{rest[:30]}» و پیدا نشد (فقط گروه‌ها
و چت رباتا).")
                    return True
                if len(found) > 1:
                    names = "\n".join(f"• {d[1][:30]}" for d in found[:6])
                    await _say(uid, ev, "🔎 چند تا پیدا شد؛ اسم دقیق‌تر بنویس:\n" + names)
                    return True
                cid, title = found[0][0], found[0][1]
            i = _pick_slot(uid, sl, explicit)
            sl[i]["chat"] = cid
            sl[i]["title"] = str(title)[:40]
            _put_slots(uid, sl)
            _cur[uid] = i
            await _say(uid, ev, f"✅ گپ اسلات {i + 1}: «{sl[i]['title']}»\n" +
_ready_text(sl[i]))
```

```
    except Exception as e:  # noqa: BLE001
        logger.info("self dot-cmd failed %s: %s", uid, e)
    return True


# ---------- پنل کاربر ----------
def _panel(uid):
    row = get_row(uid)
    sl = slots_of(row)
    on = uid in _clients
    lines = ["🤖 <b>سلف روبی</b>\n",
             f"🔌 اتصال: {'✅ وصل' if on else '❌ قطع'}",
             f"⚙️ وضعیت: {'⏸ متوقف' if row.paused else '▶️ فعال'}\n",
             "هر اسلات: کلمه رو هر چند ثانیه (تایمر) خودش توی گپ انتخابی می‌گه و بعدش دکمه‌ی انتخابی (چپ/وسط/راست) رو می‌زنه؛ لازم نیست چیزی بفرستی).",
             "💡 مثلاً توی Saved Messages): <code>.پنل</code> ، از داخل چت هم می‌تونی تنظیم کنی <code>.کلمه فلان</code> ، <code>.5 زمان</code> ، <code>.دکمه وسط</code> ، <code>.گپ</code>\n",
             "🕒 ساعت زنده کنار اسم: دکمه‌ی «ساعت» یا <code>.ساعت</code> — 🪞 روی کپی پروفایل: <code>.غیرفعال کپی</code> و برای برگشت <code>.کپی</code> پیام کاربر ریپلای کن\n"]
    kb = []
    for i in range(MAX_SLOTS):
        s = sl[i] if i < len(sl) else None
        if s and s.get("word"):
            t = f"{i + 1}⃣ «{s['word']}» {(s.get('title') or '💬')[:12]} {POS[s.get('pos', 'n')]}"
        else:
            t = f"{i + 1}⃣ ➕ تنظیم کلمه"
        kb.append([B(t[:60], callback_data=f"self:s:{i}")])
    kb.append([B("🕒 ساعت" + (" ✅" if _pget(uid).get("clock_on") else ""), callback_data="self:clk")])
    kb.append([B("🔌 قطع اتصال" if on else "🔗 اتصال اکانت", callback_data="self:disc" if on else "self:conn")])
    kb.append([B("▶️ فعال‌سازی" if row.paused else "⏸ توقف موقت", callback_data="self:tog")])
    return "\n".join(lines), KB(kb)


def _slot_panel(uid, i):
    row = get_row(uid)
    sl = slots_of(row)
    s = sl[i] if i < len(sl) else {}
    t = (f"{i + 1}⃣ <b>کلمه‌ی {i + 1}</b>\n\n📝 کلمه: {s.get('word') or '—'}\n"
         f"💬 گپ: {s.get('title') or '—'}\n"
         f"🔁 تکرار خودکار: {('هر ' + fmt_delay(s.get('delay', 0))) if s.get('word') and int(s.get('delay', 0) or 0) >= MIN_AUTO else '—'}\n🎯 دکمه: {POS[s.get('pos', 'n')]}")
    kb = KB([[B("📝 کلمه", callback_data=f"self:e:{i}:word"), B("💬 گپ", callback_data=f"self:cp:{i}:0")],
             [B("⏱ تایمر", callback_data=f"self:tm:{i}"), B("🎯 دکمه", callback_data=f"self:pm:{i}")],
```

```
            [B("🗑️ حذف", callback_data=f"self:d:{i}"), B("🔙 بازگشت",
callback_data="self:home")]])
    return t, kb


def _get_slots(uid):
    row = get_row(uid)
    sl = slots_of(row)
    while len(sl) < MAX_SLOTS:
        sl.append({})
    return sl


def _put_slots(uid, sl):
    save_row(uid, slots=json.dumps(sl, ensure_ascii=False))
    row = get_row(uid)
    if uid in _clients:
        _load_runtime(uid, row)


def _allowed(uid):
    r = get_row(uid)
    return bool(r and r.enabled)


async def cmd_self(update, context):
    u = update.effective_user
    if update.effective_chat.type != "private":
        return
    if not _allowed(u.id):
        raise ApplicationHandlerStop   # برای کسی که سلف نداره هیچ پیامی نمی‌آد
    t, kb = _panel(u.id)
    await update.effective_message.reply_text(t, reply_markup=kb, parse_mode="HTML")


async def cb_user(update, context):
    q = update.callback_query
    uid = q.from_user.id
    if not _allowed(uid):
        await q.answer()
        return
    p = q.data.split(":")
    a = p[1]
    ud = context.user_data
    await q.answer()
    if not (a == "e" and p[3] == "word"):
        _capture.pop(uid, None)

    async def home():
        ud.pop("self_st", None)
```

```
        t, kb = _panel(uid)
        await q.message.edit_text(t, reply_markup=kb, parse_mode="HTML")

    if a == "home":
        await home()
    elif a == "tog":
        r = get_row(uid)
        save_row(uid, paused=not r.paused)
        if uid in _clients:
            _load_runtime(uid, get_row(uid))
        await home()
    elif a == "s":
        i = int(p[2])
        t, kb = _slot_panel(uid, i)
        await q.message.edit_text(t, reply_markup=kb, parse_mode="HTML")
    elif a == "clk":
        tx, kb = _clock_panel(uid)
        await q.message.edit_text(tx, reply_markup=kb, parse_mode="HTML")
    elif a in ("clkt", "clkf"):
        if uid not in _clients:
            await q.answer("اول اکانتت رو وصل کن.", show_alert=True)
            return
        if a == "clkt":
            res = await clock_set(uid, on=not _pget(uid).get("clock_on"))
        else:
            res = await clock_set(uid, on=True, font=int(p[2]))
        if res.startswith(("⚠️", "❌")):
            await q.answer(res[:180], show_alert=True)
        tx, kb = _clock_panel(uid)
        try:
            await q.message.edit_text(tx, reply_markup=kb, parse_mode="HTML")
        except Exception:
            pass
    elif a == "tm":
        i = int(p[2])
        pre = [5, 10, 30, 60, 300, 600]
        kb = [[B(fmt_delay(x), callback_data=f"self:t:{i}:{x}") for x in pre[:3]],
              [B(fmt_delay(x), callback_data=f"self:t:{i}:{x}") for x in pre[3:]],
              [B("✏️ عدد دلخواه", callback_data=f"self:e:{i}:delay")], [B("🔙 بازگشت",
callback_data=f"self:s:{i}")]]
        await q.message.edit_text("⏱ هر چند ثانیه خودش کلمه رو بگه و دکمه رو بزنه؟",
reply_markup=KB(kb))
    elif a == "t":
        i, v = int(p[2]), int(p[3])
        if 0 <= v <= MAX_DELAY:
            sl = _get_slots(uid)
            sl[i]["delay"] = v
            _put_slots(uid, sl)
        tx, kb = _slot_panel(uid, i)
        await q.message.edit_text(tx, reply_markup=kb, parse_mode="HTML")
```

```
    elif a == "e" and p[3] == "word":
        i = int(p[2])
        sl = _get_slots(uid)
        if uid not in _clients:
            await q.answer("اول اکانتت رو وصل کن.", show_alert=True)
            return
        if not sl[i].get("chat"):
            await q.answer("(گپ 💬 دکمه‌ی) اول گپ رو انتخاب کن.", show_alert=True)
            return
        _capture[uid] = {"slot": i, "chat": sl[i]["chat"], "ts": time.time()}
        await q.message.edit_text(
            f"📝 الان از <b>اکانت خودت</b> توی گپ «{sl[i].get('title')}» رو کلمه‌ی دلخواهت بفرست.\n"
            "همون لحظه خودکار ثبت می‌شه؛ لازم نیست اینجا تایپ کنی. (۵ دقیقه فرصت داری)",
            parse_mode="HTML", reply_markup=KB([[B("⬅️ انصراف",
callback_data=f"self:s:{i}")]]))
    elif a == "e":
        i, f = int(p[2]), p[3]
        ud["self_st"] = {"step": f, "slot": i}
        ask = {"delay": "⏱ هر چند ثانیه خودکار بگه؟ مثلاً <code>30</code> (ثانیه) یا <code>5</code>دقیقه (حداقل ۵ ثانیه، حداکثر ۶۰ دقیقه):"}[f]
        await q.message.edit_text(ask, parse_mode="HTML", reply_markup=KB([[B("⬅️ بازگشت", callback_data=f"self:s:{i}")]]))
    elif a == "pm":
        i = int(p[2])
        await q.message.edit_text("🎯 کدوم دکمه‌ی پیام ربات زده بشه؟\n(⬜ = هیچ دکمه‌ای زده نشه)", reply_markup=KB([
            [B(POS["l"], callback_data=f"self:p:{i}:l"), B(POS["m"],
callback_data=f"self:p:{i}:m"), B(POS["r"], callback_data=f"self:p:{i}:r")],
            [B(POS["n"], callback_data=f"self:p:{i}:n")], [B("⬅️ بازگشت",
callback_data=f"self:s:{i}")]]))
    elif a == "p":
        i, v = int(p[2]), p[3]
        if v in POS:
            sl = _get_slots(uid)
            sl[i]["pos"] = v
            sl[i].setdefault("delay", 0)
            _put_slots(uid, sl)
        t, kb = _slot_panel(uid, i)
        await q.message.edit_text(t, reply_markup=kb, parse_mode="HTML")
    elif a == "cp":
        i, pg = int(p[2]), int(p[3])
        c = _clients.get(uid)
        if not c:
            await q.answer("اول اکانتت رو وصل کن.", show_alert=True)
            return
        if pg == 0 or "self_dl" not in ud:
            ud["self_dl"] = await _dialogs(c)
        pg = max(0, pg - 1)   # شماره‌ی صفحه + ۱ (۰ یعنی تازه‌سازی) = مقدار callback
        dl = ud["self_dl"]
```

```
        per = 8
        pg = max(0, min(pg, max(0, (len(dl) - 1) // per)))
        kb = [[B(("👥 " if g else "🤖 ") + n[:28], callback_data=f"self:cs:{i}:{pg *
per + k}")] for k, (cid, n, g) in enumerate(dl[pg * per:(pg + 1) * per])]
        nav = []
        if pg > 0:
            nav.append(B("◀️", callback_data=f"self:cp:{i}:{pg}"))
        if (pg + 1) * per < len(dl):
            nav.append(B("▶️", callback_data=f"self:cp:{i}:{pg + 2}"))
        if nav:
            kb.append(nav)
        kb.append([B("🔙 بازگشت", callback_data=f"self:s:{i}")])
        await q.message.edit_text("💬 (گروه‌ها و چت ربات‌های دیگه) کلمه توی کدوم گپ گفته بشه؟" if
dl else "گپی پیدا نشد.", reply_markup=KB(kb))
    elif a == "cs":
        i, k = int(p[2]), int(p[3])
        dl = ud.get("self_dl") or []
        if k >= len(dl):
            await q.answer("لیست قدیمی شده؛ دوباره باز کن.", show_alert=True)
            return
        sl = _get_slots(uid)
        sl[i]["chat"], sl[i]["title"] = dl[k][0], dl[k][1][:40]
        sl[i].setdefault("delay", 0)
        _put_slots(uid, sl)
        t, kb = _slot_panel(uid, i)
        await q.message.edit_text("✅ گپ انتخاب شد.\n\n" + t, reply_markup=kb,
parse_mode="HTML")
    elif a == "d":
        i = int(p[2])
        sl = _get_slots(uid)
        sl[i] = {}
        _put_slots(uid, sl)
        await home()
    elif a == "conn":
        if not configured():
            await q.message.edit_text("⚠️ سلف هنوز روی سرور راه‌اندازی نشده (SELF_API_ID /
SELF_API_HASH). به پشتیبانی بگو.",
                                      reply_markup=KB([[B("🔙",
callback_data="self:home")]]))
            return
        ud["self_st"] = {"step": "phone"}
        await q.message.edit_text(
            "🔗 <b>اتصال اکانت</b>\n\n⚠️ سلف از اکانت شخصی خودت استفاده می‌کنه؛ ریسکش با خودته و "
روی روباهیو کار نمی‌کنه. "
            "دکمه‌ی «اشتراک شماره من» 👇 رو بزن تا سشن کامل بسته بشه.\n\nهر وقت خواستی «قطع اتصال» رو از منوی پایین بزن.",
            parse_mode="HTML", reply_markup=KB([[B("🔙 بازگشت",
callback_data="self:home")]]))
        await q.message.reply_text("📱 شماره‌ی خودت رو با دکمه‌ی پایین بفرست:",
reply_markup=ReplyKeyboardMarkup(
```

```
            [[KeyboardButton("📱 اشتراک شماره من", request_contact=True)],
[KeyboardButton("لغو")]], resize_keyboard=True, one_time_keyboard=True))
    elif a in ("k", "kx", "ky", "kc"):
        st = ud.get("self_st")
        if not st or st.get("step") != "code":
            return
        code = ud.get("self_code", "")
        if a == "kc":
            ud.pop("self_st", None)
            ud.pop("self_code", None)
            await _drop_login(uid)
            await q.message.edit_text("لغو شد.")
            return
        if a == "k" and len(code) < 8:
            code += p[2]
        elif a == "kx":
            code = code[:-1]
        elif a == "ky":
            if len(code) < 5:
                await context.bot.send_message(uid, "❗ کد حداقل ۵ رقمه.")
                return
            await q.message.edit_text("⏳ در حال بررسی کد...")
            ud["self_code"] = ""
            await _sign_in(uid, context, code=code)
            return
        ud["self_code"] = code
        await _send_keypad(context.bot, uid, context, edit_msg=q.message)
    elif a == "disc":
        await q.message.edit_text("🔌 اتصال قطع بشه و سشن اکانتت کامل بسته بشه؟",
                                  reply_markup=KB([[B("✅ بله، قطع کن",
callback_data="self:discy"), B("لغو", callback_data="self:home")]]))
    elif a == "discy":
        await stop_client(uid, logout=True, restore=True)
        save_row(uid, session_enc=None, phone=None)
        await home()


async def _drop_login(uid):
    d = _login.pop(uid, None)
    if d:
        try:
            await d["client"].disconnect()
        except Exception:
            pass



def _keypad_kb(n):
    rows = [[B(str(d), callback_data=f"self:k:{d}") for d in r] for r in ("123",
"456", "789")]
```

```
    rows.append([B("⌫", callback_data="self:kx"), B("0", callback_data="self:k:0"),
B("✅ تأیید", callback_data="self:ky")])
    rows.append([B("❌ انصراف", callback_data="self:kc")])
    return KB(rows)


async def _send_keypad(bot, uid, context, edit_msg=None):
    code = context.user_data.get("self_code", "")
    dots = " ".join(["●"] * len(code) + ["○"] * max(0, 5 - len(code)))
    tx = f"🔢 کد تلگرامی که برات اومد رو با دکمه‌های زیر وارد کن (کد رو داخل چت تایپ نکن):\n\n<b>
{dots}</b>"
    if edit_msg:
        await edit_msg.edit_text(tx, parse_mode="HTML",
reply_markup=_keypad_kb(len(code)))
    else:
        await bot.send_message(uid, tx, parse_mode="HTML",
reply_markup=_keypad_kb(len(code)))


async def _sign_in(uid, context, code=None, password=None):
    ud = context.user_data
    bot = context.bot
    d = _login.get(uid)
    if not d or time.time() - d["ts"] > 600:
        ud.pop("self_st", None)
        ud.pop("self_code", None)
        await _drop_login(uid)
        await bot.send_message(uid, "⌛ زمان اتصال تموم شد؛ دوباره از /self بزن اتصال.")
        return
    from telethon import errors
    try:
        if password is None:
            await d["client"].sign_in(d["phone"], code, phone_code_hash=d["hash"])
        else:
            await d["client"].sign_in(password=password)
    except errors.SessionPasswordNeededError:
        ud["self_st"] = {"step": "pw"}
        await bot.send_message(uid, "🔐 تأیید دومرحله‌ای داری؛ رمزش رو بفرست (پیامت بعد از خوندن
پاک می‌شه).")
        return
    except (errors.PhoneCodeInvalidError, errors.PasswordHashInvalidError):
        if password is None:
            ud["self_code"] = ""
            await bot.send_message(uid, "❌ کد اشتباهه؛ دوباره وارد کن.")
            await _send_keypad(bot, uid, context)
        else:
            await bot.send_message(uid, "❌ رمز اشتباهه؛ دوباره بفرست (یا لغو).")
        return
    except errors.PhoneCodeExpiredError:
        ud.pop("self_st", None)
```

```
        ud.pop("self_code", None)
        await _drop_login(uid)
        await bot.send_message(uid, "⌛ کد منقضی شد؛ از /self سریع دوباره اتصال بزن و کد جدید رو با شماره‌گیر وارد کن.")
        return
    except Exception as e:  # noqa: BLE001
        logger.warning("self sign_in: %s", e)
        await bot.send_message(uid, "⚠️ خطا: " + str(e)[:150])
        return
    save_row(uid, session_enc=enc(d["client"].session.save()), phone=d["phone"])
    await _drop_login(uid)
    ud.pop("self_st", None)
    ud.pop("self_code", None)
    ok = await start_client(uid)
    tx, kb = _panel(uid)
    await bot.send_message(uid, ("✅ اکانت وصل شد!\n\n" if ok else "⚠️ ذخیره شد ولی اتصال بالا نیومد.\n\n") + tx,
                           reply_markup=kb, parse_mode="HTML")


async def user_text(update, context):
    msg = update.effective_message
    u = update.effective_user
    if not msg or not u or update.effective_chat.type != "private" or not msg.text:
        return
    text = msg.text.strip()
    ud = context.user_data
    # --- ورودی‌های مدیر ---
    aa = ud.get("admin_action")
    if isinstance(aa, str) and aa.startswith("self_") and u.id in ADMIN_IDS:
        await _admin_input(update, context, aa, text)
        raise ApplicationHandlerStop
    if text == "سلف":
        await cmd_self(update, context)   # برای کاربر بدون سلف ساکت است
        raise ApplicationHandlerStop
    st = ud.get("self_st")
    if not st:
        return
    uid = u.id
    if not _allowed(uid):
        ud.pop("self_st", None)
        return
    raise_stop = True
    try:
        if text in ("لغو", "/cancel"):
            ud.pop("self_st", None)
            await _drop_login(uid)
            await msg.reply_text("لغو شد.", reply_markup=ReplyKeyboardRemove())
        elif st["step"] == "delay":
            i, f = st["slot"], st["step"]
```

```
            sl = _get_slots(uid)
            if f == "delay":
                v = parse_delay(text)
                if v is None or 0 < v < MIN_AUTO:
                    await msg.reply_text("❗ عدد درست بفرست؛ مثلاً 30 یا 5 دقیقه (حداقل ۵ ثانیه، حداکثر ۶۰ دقیقه).")
                    raise ApplicationHandlerStop
            else:
                v = text[:40]
            sl[i][f] = v
            sl[i].setdefault("delay", 0)
            _put_slots(uid, sl)
            ud.pop("self_st", None)
            t, kb = _slot_panel(uid, i)
            await msg.reply_text("✅ ذخیره شد.\n\n" + t, reply_markup=kb,
parse_mode="HTML")
        elif st["step"] == "phone":
            await msg.reply_text("📱 شماره رو تایپ نکن؛ فقط با دکمه‌ی «اشتراک شماره من» از منوی پایین بفرست.", reply_markup=ReplyKeyboardMarkup(
                [[KeyboardButton("📱 اشتراک شماره من", request_contact=True)],
[KeyboardButton("لغو")]], resize_keyboard=True, one_time_keyboard=True))
        elif st["step"] == "code":
            try:
                await msg.delete()   # اگه کد رو تایپ کرد، فوراً پاک می‌شه
            except Exception:
                pass
            await msg.reply_text("🔢 کد رو تایپ نکن (تلگرام باطلش می‌کنه)؛ با شماره‌گیر زیر وارد کن 👇")
            await _send_keypad(context.bot, uid, context)
        elif st["step"] == "pw":
            await _sign_in(uid, context, password=text)
            try:
                await msg.delete()
            except Exception:
                pass
    except ApplicationHandlerStop:
        raise
    except Exception as e:  # noqa: BLE001
        logger.warning("self user_text: %s", e)
        await msg.reply_text("⚠️ خطا: " + str(e)[:150])
    if raise_stop:
        raise ApplicationHandlerStop


async def on_contact(update, context):
    msg = update.effective_message
    u = update.effective_user
    if not msg or not msg.contact or not u or update.effective_chat.type !=
"private":
        return
```

```
    st = context.user_data.get("self_st")
    if not st or st.get("step") != "phone" or not _allowed(u.id):
        return
    c = msg.contact
    if c.user_id != u.id:   # فقط شماره‌ی خودِ کاربر؛ مخاطب دیگران پذیرفته نمی‌شه
        await msg.reply_text("❌ بفرست «اشتراک شماره من» دکمه‌ی با خودت شماره‌ی فقط.")
        raise ApplicationHandlerStop
    phone = "+" + re.sub(r"\D", "", c.phone_number or "")
    try:
        from telethon import TelegramClient
        from telethon.sessions import StringSession
        await _drop_login(u.id)
        client = TelegramClient(StringSession(), API_ID, API_HASH)
        await client.connect()
        sent = await client.send_code_request(phone)
        _login[u.id] = {"client": client, "phone": phone, "hash":
sent.phone_code_hash, "ts": time.time()}
        st["step"] = "code"
        context.user_data["self_code"] = ""
        await msg.reply_text("📨 اومد برات تلگرام کد (چت تلگرام، اپ توی «Telegram»).",
reply_markup=ReplyKeyboardRemove())
        await _send_keypad(context.bot, u.id, context)
    except Exception as e:  # noqa: BLE001
        logger.warning("self contact: %s", e)
        await msg.reply_text("⚠️ خطا: " + str(e)[:150],
reply_markup=ReplyKeyboardRemove())
    raise ApplicationHandlerStop


# ---------- پنل مدیر ----------
ADMIN_IDS = set()


def admin_menu_kb():
    return KB([[B("➕ فعال‌سازی برای کاربر", callback_data="selfa:on")],
               [B("⏹️ غیرفعال‌سازی کاربر", callback_data="selfa:off")],
               [B("🗑️ حذف کامل (خروج از سشن)", callback_data="selfa:del")],
               [B("📋 لیست کاربران سلف", callback_data="selfa:list")],
               [B("🔙 بازگشت", callback_data="admin:back")]])


async def cb_admin(update, context):
    q = update.callback_query
    if q.from_user.id not in ADMIN_IDS:
        await q.answer("دسترسی نداری.", show_alert=True)
        return
    a = q.data.split(":")[1]
    await q.answer()
    if a == "menu":
        context.user_data.pop("admin_action", None)
```

```
        await q.message.reply_text("🤖 مدیریت سلف" + ("" if configured() else "\n\n⚠️
SELF_API_ID / SELF_API_HASH تنظیم نشده."),
                                   reply_markup=admin_menu_kb())
    elif a in ("on", "off", "del"):
        context.user_data["admin_action"] = "self_" + a
        await q.message.reply_text("🆔 آیدی عددی کاربر رو بفرست:")
    elif a == "list":
        s = get_session()
        try:
            rows = s.query(SelfAccess).all()
            lines = [f"{'✅' if r.enabled else '🔲'} <code>{r.user_id}</code> {'🔗'
if r.user_id in _clients else '—'}" for r in rows]
        finally:
            s.close()
        await q.message.reply_text("📋 کاربران سلف (✅ مجاز | 🔗 وصل):\n\n" +
("\n".join(lines) if lines else "خالیه."),
                                   parse_mode="HTML", reply_markup=admin_menu_kb())


async def _admin_input(update, context, action, text):
    context.user_data.pop("admin_action", None)
    context.user_data["ai_skip_msg"] = update.message.message_id
    t = fa2en(text).strip()
    if not t.isdigit():
        context.user_data["admin_action"] = action
        await update.message.reply_text("❗ فقط آیدی عددی بفرست.")
        return
    uid = int(t)
    bot = context.bot
    if action == "self_on":
        save_row(uid, enabled=True)
        if get_row(uid).session_enc:
            await start_client(uid)
        await update.message.reply_text(f"✅ سلف برای {uid} فعال شد.",
reply_markup=admin_menu_kb())
        try:
            await bot.send_message(uid, "✅ برات فعال کرد «سلف» رو.\nپشتیبانی، برای تنظیم /self
رو بزن یا بنویس «سلف».")
        except Exception:
            pass
    elif action == "self_off":
        if not get_row(uid):
            await update.message.reply_text("❌ این کاربر سلف نداره.",
reply_markup=admin_menu_kb())
            return
        save_row(uid, enabled=False)
        await stop_client(uid, restore=True)
        await update.message.reply_text(f"🔲 سلف {uid} غیرفعال شد.",
reply_markup=admin_menu_kb())
    else:
```

```
        r = get_row(uid)
        if not r:
            await update.message.reply_text("❌ این کاربر سلف نداره.",
reply_markup=admin_menu_kb())
            return
        await stop_client(uid, logout=True, restore=True)
        s = get_session()
        try:
            x = s.get(SelfAccess, uid)
            if x:
                s.delete(x)
            y = s.get(SelfProfile, uid)
            if y:
                s.delete(y)
            s.commit()
        finally:
            s.close()
        await update.message.reply_text(f"🗑️ سلف {uid} کامل حذف شد.",
reply_markup=admin_menu_kb())


def register(app, admin_ids):
    ADMIN_IDS.clear()
    ADMIN_IDS.update(admin_ids)
    app.add_handler(CommandHandler("self", cmd_self), group=-5)
    app.add_handler(CallbackQueryHandler(cb_user, pattern=r"^self:"), group=-5)
    app.add_handler(CallbackQueryHandler(cb_admin, pattern=r"^selfa:"), group=-5)
    app.add_handler(MessageHandler(filters.CONTACT & filters.ChatType.PRIVATE,
on_contact), group=-5)
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND &
filters.ChatType.PRIVATE, user_text), group=-5)
```